Ministerul Educatiei si Cercetarii al Republicii Moldova

Universitatea Tehnica a Moldovei

Facultatea Calculatoare, Informatica si Microelectronica

Laboratory Work 3:

Empirical Analysis of Graph Traversal Algorithms: DFS and BFS

Elaborated: Ilie Covali
st. gr. FAF-243

Verified: asist. univ. Fistic Cristofor

Chisinau – 2026

Contents

1 ALGORITHM ANALYSIS **2**
1.1 Objective 2
1.2 Tasks 2
1.3 Theoretical Notes 2
1.4 Comparison Metric 2
1.5 Input Format 2

2 IMPLEMENTATION **3**
2.1 Breadth First Search (BFS) 3
2.1.1 Algorithm Description 3
2.1.2 Implementation 3
2.1.3 Results 5
2.2 Depth First Search (DFS) 6
2.2.1 Algorithm Description 6
2.2.2 Implementation 6
2.2.3 Results 8
2.3 Comparative Analysis 10
2.4 Edge Case Analysis 11

3 CONCLUSION **13**

1 ALGORITHM ANALYSIS

1.1 Objective

Study and analyze Breadth First Search (BFS) and Depth First Search (DFS) using empirical measurements.

1.2 Tasks

1. Implement BFS and DFS in a programming language;
2. Establish the properties of the input data against which the analysis is performed;
3. Choose metrics for comparing the algorithms;
4. Perform empirical analysis of the proposed algorithms;
5. Make a graphical presentation of the obtained data;
6. Formulate conclusions based on the performed work.

1.3 Theoretical Notes

Breadth First Search and Depth First Search are fundamental graph traversal algorithms. For a graph represented with adjacency lists, both have theoretical time complexity $O(V+E)$, where V is the number of vertices and E is the number of edges.

BFS explores the graph level by level from a start vertex using a queue. This makes it suitable for shortest paths in unweighted graphs.

DFS explores a path as deep as possible before backtracking, usually implemented with recursion or a stack. It is useful in connectivity checks, topological ordering, and cycle detection.

Although both algorithms have the same asymptotic complexity, practical runtime differs because of data structure overheads and memory access patterns.

1.4 Comparison Metric

The selected metric is average execution time in seconds for traversing each graph, measured with `time.perf_counter()` and averaged over multiple runs.

1.5 Input Format

The input for each algorithm is a connected, undirected sparse graph represented implicitly.

Input properties used in the experiments:

- Number of vertices sampled from 1000 up to 1000000 (24 input sizes);
- Each vertex i has neighbors $(i \pm 1)$, $(i \pm 2)$, $(i \pm 3)$ modulo n;
- The resulting graph is connected and has constant degree 6;
- 5 timed traversals per input size (average reported).

2 IMPLEMENTATION

Both algorithms were implemented in Python. To avoid recursion depth limitations for large graphs, DFS was implemented iteratively with an explicit stack. For scalability up to one million vertices, neighbors are computed on demand instead of storing full adjacency lists.

2.1 Breadth First Search (BFS)

2.1.1 Algorithm Description

The algorithm starts from a source node, marks it as visited, then repeatedly extracts a node from a queue and visits all of its unvisited neighbors.

Pseudocode:

```
BFS(graph, start):
    visited[start] <- true
    queue <- [start]
    while queue is not empty:
        u <- pop_front(queue)
        for each v in graph[u]:
            if not visited[v]:
                visited[v] <- true
                push_back(queue, v)
```

2.1.2 Implementation

Listing 1: BFS implementation in Python

```python
from collections import deque

DEGREE_STEPS = (1, 2, 3)

def bfs(node_count, start=0):
    node_count = int(node_count)
    if node_count <= 0:
        return 0

    start = int(start) % node_count
    visited = bytearray(node_count)
    queue = deque([start])
    visited[start] = 1
    visited_count = 0

    while queue:
        node = queue.popleft()
        visited_count += 1
        for step in DEGREE_STEPS:
            n_plus = (node + step) % node_count
            n_minus = (node - step) % node_count
            if not visited[n_plus]:
                visited[n_plus] = 1
```

```
                queue.append(n_plus)
            if not visited[n_minus]:
                visited[n_minus] = 1
                queue.append(n_minus)

    return visited_count
```

2.1.3 Results

n	avg_time_s
1000	0.000472
2000	0.000988
4000	0.002015
6000	0.003063
8000	0.004058
10000	0.005124
12000	0.006297
15000	0.008170
18000	0.009312
20000	0.010856
30000	0.015968
40000	0.021218
60000	0.032052
80000	0.041475
100000	0.051605
120000	0.064745
150000	0.085508
180000	0.096394
200000	0.106976
300000	0.160194
400000	0.243912
600000	0.320977
800000	0.414642
1000000	0.523759

Figure 1: Table of average execution times for BFS

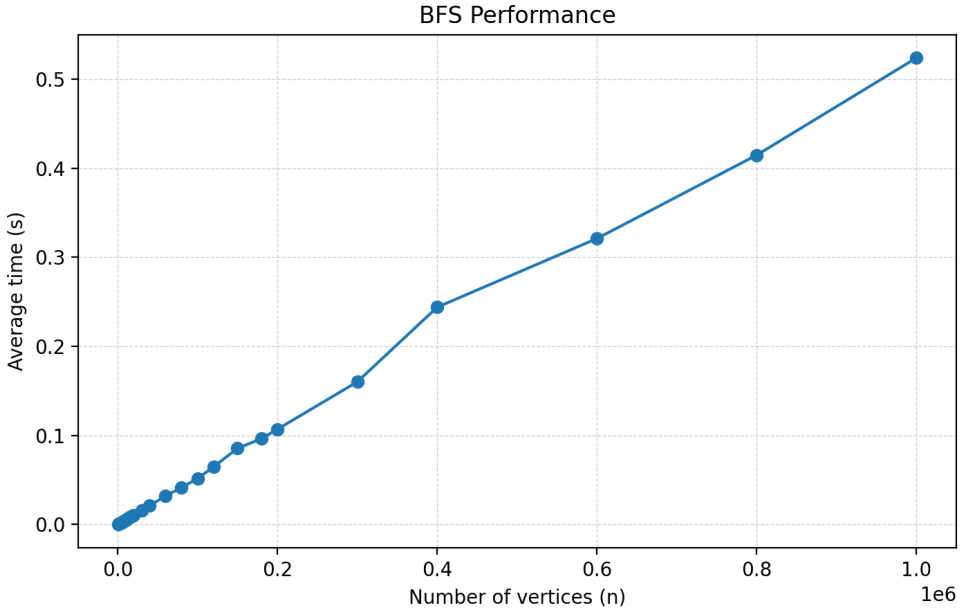

Figure 2: BFS performance graph

The BFS graph shows approximately linear growth with the number of vertices, consistent with the theoretical complexity for sparse graphs.

2.2 Depth First Search (DFS)

2.2.1 Algorithm Description

The algorithm starts from a source node and explores neighbors deeply before backtracking. An explicit stack is used to avoid recursion depth issues.

Pseudocode:

```
DFS(graph, start):
    stack <- [start]
    while stack is not empty:
        u <- pop(stack)
        if visited[u]:
            continue
        visited[u] <- true
        for each v in graph[u]:
            if not visited[v]:
                push(stack, v)
```

2.2.2 Implementation

Listing 2: DFS implementation in Python

```python
DEGREE_STEPS = (3, 2, 1)

def dfs(node_count, start=0):
    node_count = int(node_count)
    if node_count <= 0:
```

```
        return 0

    start = int(start) % node_count
    visited = bytearray(node_count)
    stack = [start]
    visited_count = 0

    while stack:
        node = stack.pop()
        if visited[node]:
            continue

        visited[node] = 1
        visited_count += 1

        for step in DEGREE_STEPS:
            n_plus = (node + step) % node_count
            n_minus = (node - step) % node_count
            if not visited[n_plus]:
                stack.append(n_plus)
            if not visited[n_minus]:
                stack.append(n_minus)

    return visited_count
```

2.2.3 Results

n	avg_time_s
1000	0.000613
2000	0.001356
4000	0.002663
6000	0.004096
8000	0.005395
10000	0.006409
12000	0.007695
15000	0.009634
18000	0.011596
20000	0.012823
30000	0.019360
40000	0.026180
60000	0.039231
80000	0.053464
100000	0.066341
120000	0.079442
150000	0.099115
180000	0.118701
200000	0.152146
300000	0.201547
400000	0.267596
600000	0.400835
800000	0.528598
1000000	0.676671

Figure 3: Table of average execution times for DFS

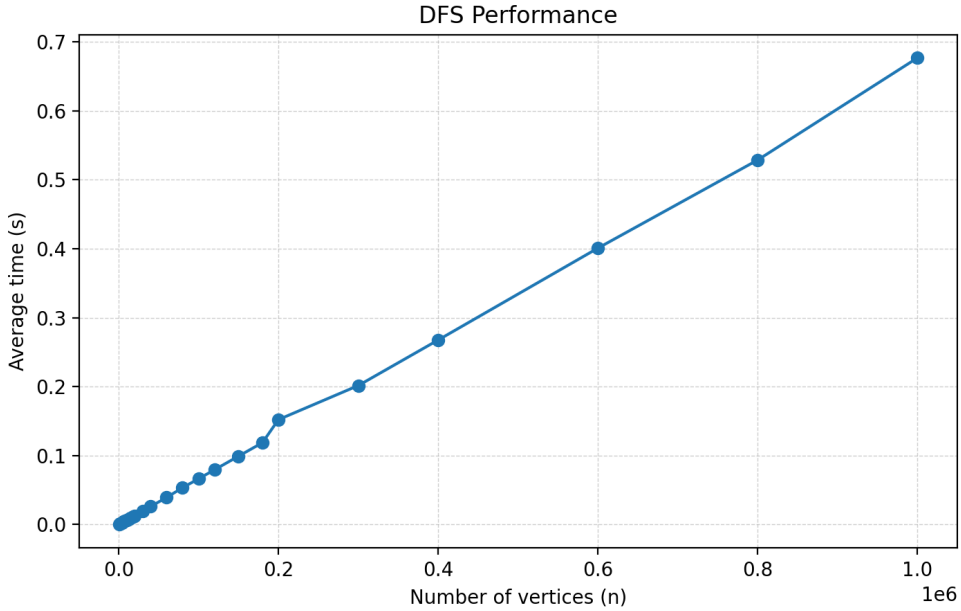

Figure 4: DFS performance graph

The DFS graph also follows a near-linear trend, confirming expected behavior for sparse constant-degree graphs.

2.3 Comparative Analysis

n	bfs_avg_time_s	dfs_avg_time_s	faster
1000	0.000472	0.000613	BFS
2000	0.000988	0.001356	BFS
4000	0.002015	0.002663	BFS
6000	0.003063	0.004096	BFS
8000	0.004058	0.005395	BFS
10000	0.005124	0.006409	BFS
12000	0.006297	0.007695	BFS
15000	0.008170	0.009634	BFS
18000	0.009312	0.011596	BFS
20000	0.010856	0.012823	BFS
30000	0.015968	0.019360	BFS
40000	0.021218	0.026180	BFS
60000	0.032052	0.039231	BFS
80000	0.041475	0.053464	BFS
100000	0.051605	0.066341	BFS
120000	0.064745	0.079442	BFS
150000	0.085508	0.099115	BFS
180000	0.096394	0.118701	BFS
200000	0.106976	0.152146	BFS
300000	0.160194	0.201547	BFS
400000	0.243912	0.267596	BFS
600000	0.320977	0.400835	BFS
800000	0.414642	0.528598	BFS
1000000	0.523759	0.676671	BFS

Figure 5: BFS vs DFS measured times for each input size

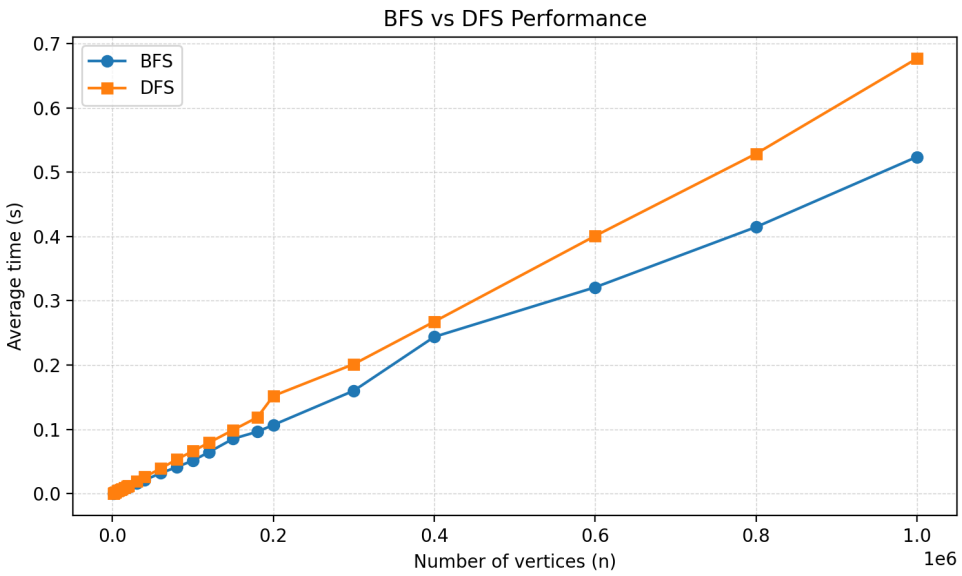

Figure 6: BFS and DFS performance comparison graph

The comparison shows that BFS was consistently faster than DFS for all tested input sizes in this implementation. At the same time, both curves remain near-linear, which confirms the same asymptotic complexity class $O(V+E)$ and highlights differences in constant factors.

2.4 Edge Case Analysis

To validate robustness, several boundary and invalid-input scenarios were tested. The following implementation details explain most outcomes:

- For non-positive sizes, both implementations return 0 immediately;
- For positive sizes, the start index is normalized with modulo n;
- Float parameters are supported through explicit normalization to integer indices.

case	n	start	bfs	dfs	note
negative_n	-5	0	0	0	Invalid size handled as empty grap
zero_n	0	0	0	0	Boundary case n=0
single_vertex	1	0	1	1	Boundary case n=1
negative_start	5	-2	5	5	Negative start is normalized modulo
start_out_of_range	5	7	5	5	Out-of-range start is normalized modu
very_large_start	5	1000000000	5	5	Very large start is normalized modulo
two_vertices	2	1	2	2	Repeated neighbors are filtered by visited
three_vertices	3	2	3	3	Small dense overlap still traverses all ve
float_n	10.5	0	10	10	Float n is supported via normalization t
float_start	10	3.7	10	10	Float start is supported via normalization

Figure 7: Edge-case behavior for BFS and DFS

The table confirms consistent behavior between BFS and DFS for boundary values ($n = 0$, $n = 1$), negative or oversized start indices, very small graphs where neighbors overlap heavily, and float parameters normalized to integer values.

3 CONCLUSION

The laboratory objective was achieved by implementing BFS and DFS, defining input properties, selecting a comparison metric, running experiments, and presenting results graphically.

The measured data validates the theoretical complexity of both algorithms for sparse graphs: runtime grows approximately linearly with problem size when the degree remains constant.

From a practical perspective, BFS and DFS are both efficient for large sparse graphs, while BFS had a lower measured constant factor in this environment. The algorithm choice should still depend on task requirements: BFS for level-order exploration and shortest paths in unweighted graphs, DFS for deep exploration and structural graph analysis.

Edge-case testing also showed that both implementations are robust at boundaries and with normalized start indices, while still rejecting unsupported input types clearly.

Sources

1. https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/
2. https://www.geeksforgeeks.org/depth-first-search-or-dfs-for-a-graph/
3. https://www.youtube.com/watch?v=zaBht0DEL0w
4. https://github.com/owhowh0/AALab3